

ULTIMATE REFERENCE GUIDE

NoSQL & MongoDB Mastery

The Complete Course Notes (A4 Format)

Optimized for PDF Printing

PDF EXPORT TIP

When printing or saving to PDF from your browser:

- Set Paper Size to **A4**
- Set Margins to **None** (the CSS handles it)
- Ensure **Background graphics** is checked

MODULE 1

NOSQL MONGODB SUMMARY

STUDY SUMMARY

NoSQL & MongoDB

Complete Conversation Recap — Concepts, Commands & Practical
Examples

Technical Reference · Compiled from Study Session

Table of Contents

1. SQL vs. NoSQL
2. ACID vs. BASE
3. The CAP Theorem
4. NoSQL Database Types
5. BSON (Binary JSON)
6. Embedding vs. Referencing
7. Mongo Shell — Basic Commands
8. Query Operators
9. The Aggregation Pipeline
10. Sort, Skip & Limit
11. find() vs. findOne()
12. Projection
13. Index Properties
14. Indexing — Pros & Cons

SQL vs. NoSQL

SQL (Relational Databases)

Examples: SQL Server, MySQL, PostgreSQL.

- **Structured** — data lives in tables with a fixed, predefined schema (columns and types).
- **Relations** — tables are linked via Foreign Keys.
- **ACID compliant** — very high accuracy in transactions (critical for banking, e-commerce transactions).
- **Vertical Scaling** — scale by upgrading a single server (RAM/CPU).
- A unified query language (SQL) across implementations.

NoSQL (Non-Relational Databases)

Example: MongoDB.

- **Unstructured / Flexible Schema** — each document in a collection can have a different shape.
- **No fixed relations** — instead of JOINS, data uses Embedding or Referencing.
- **BASE model** instead of ACID: Basically Available, Soft state, Eventually consistent — faster, less strict on immediate consistency.
- **Horizontal Scaling** — add more servers (Sharding), easier to scale with massive data volumes.
- Multiple types exist: Document (MongoDB), Key-Value (Redis), Column-family (Cassandra), Graph (Neo4j).

When to Use Which

Scenario	Better Fit
Complex, stable relationships (financial transactions)	SQL
Frequently changing data shape, fast development needs	NoSQL

Scenario	Better Fit
Massive scale (millions of users)	Usually NoSQL
Complex reports requiring many joins	SQL

ACID vs. BASE

ACID (SQL Model)

Philosophy: accuracy and consistency come first, even at the cost of speed.

Letter	Meaning	Explanation
Atomicity	All-or-nothing	A transaction either completes fully or is fully rolled back — no partial state.
Consistency	Valid state transitions	Data moves from one valid state to another; no rule/constraint violations.
Isolation	Concurrent safety	Concurrent transactions behave as though each runs alone, with no interference.
Durability	Permanence	Once committed, data is guaranteed to persist even after a crash.

Example: transferring money between two bank accounts — ACID is mandatory, since a partial transfer (money leaving one account but never arriving at the other) is unacceptable.

BASE (NoSQL Model)

Philosophy: prioritize availability and speed, accepting that data may take a moment to fully synchronize.

Letter	Meaning	Explanation
Basically Available	Always responds	The system responds to every request, even if part of it is degraded.
Soft state	Not fixed	Data state can change over time even without new input, due to replica synchronization.

Letter	Meaning	Explanation
Eventually consistent	Converges over time	Replicas aren't instantly identical, but will converge within a short time.

Example: a Like count on a social media post — a brief delay in updating the number across viewers is harmless. Speed matters more than instant accuracy.

MongoDB in Practice

MongoDB supports multi-document transactions since version 4.0, giving a choice: use ACID transactions when accuracy is essential, or rely on eventual consistency when speed matters more.

```
const session = await mongoose.startSession();
session.startTransaction();
try {
  await Order.create([orderData], { session });
  await Product.updateMany(..., { session });
  await session.commitTransaction();
} catch (err) {
  await session.abortTransaction();
}
```


The CAP Theorem

In any distributed database, a system cannot simultaneously guarantee all three of the following once a network issue occurs — it can guarantee at most two.

The Three Properties

- **Consistency (C)** — every node returns the exact same data at the same moment.
- **Availability (A)** — every request receives a response (success or failure), even if it isn't the newest data.
- **Partition Tolerance (P)** — the system keeps functioning even if nodes lose the ability to communicate with each other.

Why Not All Three?

Network partitions are inevitable in distributed systems. So the real trade-off during a partition is between Consistency and Availability — Partition Tolerance isn't optional, since network failures will happen regardless.

- **Choosing Consistency** — some nodes must stop responding (reject requests) until data is synchronized, to avoid returning incorrect data.
- **Choosing Availability** — requests are served from every node, but data may briefly differ between nodes (not fully consistent).

Real-World Examples

System	Choice	Why
MongoDB (default)	CP	Prefers to pause writes on the primary rather than risk inconsistency across replicas.
Cassandra	AP	Prefers to always respond, even with slightly stale data, then reconciles eventually.

Read/Write Concerns in MongoDB

```
// Write Concern - confirms the write reached a certain number of nodes
db.orders.insertOne(order, { writeConcern: { w: "majority" } })

// Read Concern - ensures reads come from a synchronized copy
db.orders.find().readConcern("majority")
```

Using `w: "majority"` sacrifices a bit of speed to guarantee higher consistency — CAP trade-offs are configuration decisions you make directly in your project.

NoSQL Database Types

1. Key-Value Store

The simplest type — every entry is a unique key mapped to a value (string, number, JSON, binary). Like a distributed HashMap/Dictionary.

Examples: Redis, Riak. *Use cases:* caching, session storage, real-time leaderboards, temporary shopping carts.

2. Wide Column Store

Similar to tables, but each row can have different columns from the next. Data is stored column-by-column rather than row-by-row, making it very fast for reading specific columns across millions of rows.

Examples: Apache HBase, Cassandra. *Use cases:* Big Data analytics, time-series data (IoT sensors, logs), systems requiring massive high-speed writes.

3. Document Store

Data is stored as documents (typically JSON/BSON), each with a flexible structure.

Examples: MongoDB, CouchDB. *Use cases:* general web/mobile applications, variable-shaped content (CMS, catalogs) — the most common NoSQL type in MERN-stack projects.

4. Graph Store

Data is represented as Nodes (entities) and Edges (relationships), designed specifically to query complex relationships quickly — something SQL and MongoDB struggle with.

Examples: Neo4j, InfiniteGraph. *Use cases:* social networks (friend recommendations), recommendation engines, fraud detection, knowledge graphs.

Quick Comparison

Type	Strongest At	Weakest At
Key-Value	Lookup by key	Complex queries

Type	Strongest At	Weakest At
Wide Column	Reading/writing single columns at massive scale	Complex relationships
Document	Flexibility + medium-complexity queries	Many joins between collections
Graph	Relationships and traversal	Simple flat data at huge scale

BSON (Binary JSON)

BSON stands for Binary JSON — the format MongoDB actually uses to store documents on disk, not plain JSON.

Why Not Plain JSON?

Problem with JSON	BSON's Solution
No precise data types (every number is just "number")	Specific types: Int32, Int64, Double, Decimal128
No dedicated date type	Native Date type stored directly as binary
No binary data type (images, files)	Native Binary Data type
Slow parsing (must be read as text first)	Faster traversal — field lengths are known upfront without reading the entire string
No ObjectId type	ObjectId is a native BSON type (used for MongoDB's <code>_id</code>)

Important Note

The JSON you see in a response (Postman, frontend, etc.) is a textual representation automatically converted from BSON. BSON itself is stored as binary inside MongoDB's `.wt` files (WiredTiger storage engine).

The 16MB Document Limit

This is a direct consequence of BSON's design: it uses an Int32 to store document size, capping the maximum possible size at 16MB. This is why arrays that can grow unbounded (e.g., a huge number of comments) should use Referencing instead of Embedding.

Embedding vs. Referencing

The most important design decision in MongoDB schema design.

Embedding

Nesting related data directly inside the parent document.

```
{
  name: "Ashraf",
  addresses: [
    { city: "Shebin El-Kom", street: "..."}
  ]
}
```

Use it when the relationship is "one-to-few" and the data is almost always read together.

Referencing

Similar to a Foreign Key — storing a reference (ID) to another document instead of embedding it.

```
{
  name: "Ashraf",
  projectId: ObjectId("...")
}
```

Use it when the relationship is "one-to-many" at scale, or the data changes frequently, or it's shared across multiple parents.

Golden Rule: if an array can grow without bound (e.g., comments on a viral post), never embed it — MongoDB documents are capped at 16MB.

Mongo Shell — Basic Commands

Connecting & Navigation

```
mongosh
mongosh "mongodb://localhost:27017/mydb"
mongosh "mongodb+srv://cluster.mongodb.net/mydb" -u user -p
```

```
show dbs
use myDatabase
db
show collections
```

Insert

```
db.projects.insertOne({ title: "GoRent" })
db.projects.insertMany([{ title: "GoRent" }, { title: "Portfolio" }])
```

Find

```
db.projects.find()
db.projects.findOne({ title: "GoRent" })
db.projects.find({ createdAt: { $gt: new Date("2025-01-01") } })
db.projects.find({}, { title: 1, techStack: 1, _id: 0 })
```

Update

```
db.projects.updateOne({ title: "GoRent" }, { $set: { status: "completed" } })
db.products.updateOne({ _id: id }, { $inc: { stock: -1 } })
db.projects.updateOne({ _id: id }, { $push: { challenges: "New challenge" } })
```

Delete

```
db.projects.deleteOne({ title: "GoRent" })
db.projects.deleteMany({ status: "archived" })
db.projects.drop()
```

Indexes

```
db.projects.createIndex({ title: 1 })
db.projects.getIndexes()
db.projects.dropIndex("title_1")
```

User Management

```
use admin
db.createUser({
  user: "ashraf",
  pwd: "securePassword",
  roles: [{ role: "readWrite", db: "myapp" }]
})
```


Query Operators

Comparison Operators

```
db.products.find({ price: { $gt: 200 } })
db.products.find({ price: { $gte: 200 } })
db.products.find({ price: { $lt: 500 } })
db.products.find({ price: { $lte: 500 } })
db.products.find({ price: { $ne: 100 } })
db.products.find({ category: { $in: ["Electronics", "Books"] } })
db.products.find({ category: { $nin: ["Electronics", "Books"] } })
```

Logical Operators

```
// $and – all conditions must be true
db.posts.find({ $and: [{ likes: { $gt: 200 } }, { likes: { $lt: 500 } }] })

// Simpler equivalent when on the same field
db.posts.find({ likes: { $gt: 200, $lt: 500 } })

// $or – at least one condition true
db.posts.find({ $or: [{ likes: { $gt: 500 } }, { category: "Trending" }] })

// $nor – neither condition true
db.posts.find({ $nor: [{ likes: { $gt: 500 } }, { category: "Trending" }] })
```

Element Operators

```
db.products.find({ discount: { $exists: true } })
db.products.find({ price: { $type: "double" } })
```

Array Operators

```
db.projects.find({ techStack: { $all: ["Next.js", "TypeScript"] } })
db.projects.find({ techStack: { $size: 3 } })
db.orders.find({ items: { $elemMatch: { productId: id, quantity: { $gte: 2 } } } })
```

Evaluation Operators

```
db.products.find({ title: { $regex: "^Laptop", $options: "i" } })
db.products.find({ $expr: { $gt: ["$price", "$discountPrice"] } })
db.products.find({ price: { $mod: [5, 0] } })
```

The Aggregation Pipeline

Core Stages

```
// $match – filtering, like WHERE
{ $match: { status: "completed", totalPrice: { $gte: 100 } } }

// $group – grouping, like GROUP BY
{ $group: {
  _id: "$userId",
  totalSpent: { $sum: "$totalPrice" },
  orderCount: { $sum: 1 },
  avgOrder: { $avg: "$totalPrice" }
}}

// $sort / $limit / $skip
{ $sort: { price: -1 } }
{ $skip: 10 }
{ $limit: 5 }

// $project – reshaping fields, like SELECT
{ $project: { title: 1, price: 1, priceAfterTax: { $multiply: ["$price", 1.14] }, _id: 0 } }

// $lookup – join across collections
{ $lookup: { from: "products", localField: "productId", foreignField: "_id",
as: "productDetails" } }

// $unwind – flatten an array into separate documents
{ $unwind: "$items" }

// $addFields – add computed fields without removing existing ones
{ $addFields: { finalPrice: { $subtract: ["$price", "$discount"] } } }

// $count
{ $count: "totalElectronics" }
```

Advanced Stages

```
// $facet – run multiple sub-pipelines at once (useful for dashboards)
{ $facet: {
  priceStats: [{ $group: { _id: null, avgPrice: { $avg: "$price" } } }],
  categoryCounts: [{ $group: { _id: "$category", count: { $sum: 1 } } } ]
}}

// $bucket – group data into ranges
{ $bucket: { groupBy: "$price", boundaries: [0, 100, 500, 1000], default:
"Other" } }

// $replaceRoot – reshape the root document
{ $replaceRoot: { newRoot: "$items" } }

// $out / $merge – write pipeline results to a collection
{ $merge: { into: "userTotals", whenMatched: "merge" } }

// $sample – random documents
{ $sample: { size: 3 } }
```

Common Group/Project Operators

Operator	Purpose
\$sum	Sum values
\$avg	Average
\$max / \$min	Highest / lowest value
\$push	Collect values into an array
\$addToSet	Like \$push, but without duplicates
\$first / \$last	First / last document in the group
\$cond	If/else expression

Operator	Purpose
\$ifNull	Default value if null

Sort, Skip & Limit

sort()

```
db.products.find().sort({ price: 1 })    // ascending
db.products.find().sort({ price: -1 })   // descending
db.products.find().sort({ category: 1, price: -1 }) // multi-field
```

limit() & skip()

```
db.products.find().limit(5)
db.products.find().skip(10)
```

Pagination — All Three Together

```
const page = 2;
const limitPerPage = 10;

db.products.find()
  .sort({ createdAt: -1 })
  .skip((page - 1) * limitPerPage)
  .limit(limitPerPage);
```

Order matters: always sort first, then skip, then limit — otherwise pagination results can be inconsistent across pages.

Mongoose Example

```
const products = await Product.find({ category: "Electronics" })
  .sort({ ratingsAverage: -1 })
  .skip((page - 1) * limit)
  .limit(limit);
```


find() vs. findOne()

find()

Returns a cursor pointing to every document matching the filter — could be zero, one, or millions of documents.

```
db.products.find({ category: "Electronics" })
```

- Returns an array-like cursor, not a plain array.
- Supports chaining `.sort()`, `.limit()`, `.skip()`.
- Returns an empty cursor (not null) if nothing matches.

findOne()

Returns only the first matching document, even if multiple documents match.

```
db.products.findOne({ category: "Electronics" })
```

- Returns a single object directly (not an array, not a cursor).
- Returns `null` if no match is found.

When to Use Which

Scenario	Use
Need a full list (e.g., all products in a category)	<code>find()</code>
Check if a document exists (e.g., email lookup at signup)	<code>findOne()</code>
Fetch a user by a unique field (<code>_id</code> , email)	<code>findOne()</code> / <code>findById()</code>
Pagination / paged listings	<code>find()</code>

Mongoose adds `findById()` as shorthand for `findOne({ _id: id })`.

Topic Eleven — find() vs. findOne()

Projection

The second parameter of `find()`, used to control which fields are returned instead of the entire document.

```
db.Posts.find(  
  { Sticker: { $nin: ["Happy Bear", "Angry Bear"] } },  
  { UID: 1, Comments: 1 }  
)
```

Inclusion (value: 1)

Specify only the fields you want; everything else is excluded automatically.

```
db.Posts.find({}, { UID: 1, Comments: 1 })
```

Exclusion (value: 0)

Specify fields to exclude; everything else is returned.

```
db.Posts.find({}, { Sticker: 0 })
```

Excluding `_id`

```
db.Posts.find({}, { UID: 1, Comments: 1, _id: 0 })
```

Important rule: you cannot mix inclusion and exclusion in the same query, except for `_id`, which is always allowed as an exception.

Practical Example

```
db.products.find(  
  { category: "Electronics" },  
  { title: 1, price: 1, imageCover: 1, ratingsAverage: 1 }  
)
```

Reduces the amount of data transferred over the network and speeds up response time, especially with large datasets.

Index Properties

1. Unique

Prevents duplicate values in a field across all documents.

```
db.users.createIndex({ email: 1 }, { unique: true })
```

Use case: email or username fields in an auth system, to prevent duplicate account registration.

2. Sparse

The index ignores documents where the field is missing entirely, instead of indexing them as null.

```
db.users.createIndex({ phoneNumber: 1 }, { sparse: true })
```

Use case: optional fields (e.g., not every user provides a phone number). Combining sparse with unique avoids conflicts that would otherwise occur when many documents share a null value.

3. Partial

Similar to sparse but more precise — lets you define a custom filter expression, so the index only covers documents matching that condition.

```
db.orders.createIndex(  
  { customerId: 1 },  
  { partialFilterExpression: { status: "completed" } }  
)
```

Use case: indexing only completed orders instead of all orders, reducing index size and speeding up frequently filtered queries.

4. Compound

A single index built from more than one field.

```
db.products.createIndex({ category: 1, price: -1 })
```

Use case: filtering products by category while sorting by price simultaneously — much faster than two separate indexes.

The ESR Rule (Equality, Sort, Range) determines the best field order inside a compound index: equality fields first, then sort fields, then range fields.

Indexing — Pros & Cons

Pros

Benefit	Explanation
Read performance	Instead of a full collection scan, the index jumps directly to the right location.
Faster sorting	If an index exists on the sort field, MongoDB can use the index's existing order instead of sorting all data at query time.
Improved aggregation	Early \$match/\$sort stages benefit from indexes, reducing documents processed in later stages.
Enforcing data rules (unique)	A unique index prevents duplicate values (e.g., email) without extra application-level validation.
Reduced memory pressure	Index lookups load only a small portion of data into memory compared to scanning the full collection.

Cons

Drawback	Explanation
Slower writes	Every insert, update, or delete must update all related indexes — more indexes mean slower writes.
Extra storage usage	Each index takes up disk space separate from the actual data.
RAM consumption	MongoDB tries to keep active indexes in memory; if indexes exceed available RAM, disk I/O slows things down instead of helping.
Maintenance complexity	Requires ongoing monitoring to identify and remove unused indexes.

Drawback	Explanation
Hard limit on count	MongoDB allows a maximum of 64 indexes per collection.
Misuse can hurt performance	A poorly chosen index (rarely filtered field, wrong field order in a compound index) wastes space and resources with little benefit.

Golden Rule

Do index: fields frequently used in `find()/$match` conditions, `sort()`, and foreign-key-like references.

Don't index: rarely filtered/sorted fields, or fields that change value extremely frequently (e.g., counters updated every second), since the write overhead outweighs the benefit.

Practical Example

```
db.products.createIndex({ category: 1, price: -1 }) // frequently filtered
and sorted
db.reviews.createIndex({ productId: 1 })           // foreign-key-like,
used in $lookup
```

MODULE 2

NOSQL DATABASE DESIGN

STUDY GUIDE

NoSQL Database Design

Foundations, Design Patterns & the Schema Design Process

Technical Reference · MongoDB Schema Design

Table of Contents

1. SQL Mindset vs. NoSQL Mindset
2. Design Pattern 1 — Embedding
3. Design Pattern 2 — Referencing
4. Design Pattern 3 — Subset Pattern
5. Design Pattern 4 — Bucket Pattern
6. Design Pattern 5 — Computed Pattern
7. Relationship Types
8. The Schema Design Process (Steps 1–4)
9. Step 5 — Avoiding Common Mistakes
10. Step 6 — Auditing a Real Project

SQL Mindset vs. NoSQL Mindset

SQL: Think "Normalization" First

The philosophy: split data into the smallest possible units, avoid any duplication, and link everything with Foreign Keys. This decision is made while designing the tables — often before you even know which queries will run against them.

```
-- SQL: three separate tables, linked by Foreign Keys
Users (id, name, email)
Orders (id, user_id, date)
OrderItems (id, order_id, product_id, quantity)
```

Whenever a query needs combined data, you run a **JOIN** at query time.

NoSQL: Think "Query Patterns" First

The philosophy: design the schema around how you'll read the data, not around how "clean" it looks in theory. The question isn't "what's the correct relationship between the data?" — it's **"what query will run most often, and how do I make it fast?"**

```
// NoSQL: order data and its related product info are grouped together
{
  _id: 1,
  user: { name: "Ashraf", email: "..."}, // Embedded
  date: "...",
  items: [
    { productName: "Laptop", quantity: 1, price: 15000 }
  ]
}
```

No JOIN needed at read time — everything is already available in a single document.

The Core Difference

	SQL	NoSQL
Starting point	Relationships between entities	Actual queries that will run (access patterns)
Primary goal	Avoid data duplication (integrity)	Read performance
Schema flexibility	Fixed, hard to migrate	Flexible — add fields to any document independently
Decision timing	Tables designed first, queries come later	Queries must be known first, then the schema is designed

Most common mistake: bringing the full SQL mindset into NoSQL — referencing everything like Foreign Keys and joining (\$lookup) on every query. The result is poor performance, because you lose NoSQL's biggest advantage: keeping related data together from the start.

TIP

- Golden rule: "Data that is accessed together should be stored together."
- Before writing a single line of schema code, write down the top 5–10 queries your app will actually run.
- If you find yourself justifying a schema decision with "because that's how it'd work in SQL," stop and re-check against your access patterns.

Design Pattern 1 — Embedding

The idea: place related data directly inside the same document, instead of a separate collection.

```
{
  _id: 1,
  name: "Ashraf",
  addresses: [
    { city: "Shebin El-Kom", street: "Main St", isDefault: true },
    { city: "Cairo", street: "Nasr City", isDefault: false }
  ]
}
```

When to Use It

- The relationship is **one-to-few** (e.g., a user has 2–3 addresses, not thousands).
- The related data is **almost always read together** (there's no scenario where you fetch the user without their addresses).
- The data **doesn't change frequently or independently** of the parent.

Pros

- A single query retrieves everything (no \$lookup needed).
- Excellent read performance.
- Atomic updates — parent and embedded data update together in one operation.

Cons

- If the array grows too large, the document size grows with it (risking the 16MB limit).
- If the embedded data is shared across multiple parents, duplication occurs.

TIPS & TRICKS

- A good rule of thumb: embed when the array size is bounded and predictable (rarely exceeds a few dozen items).
- If you're unsure whether to embed, ask: "Will I ever need this child data without its parent?" If no — embed.
- Embedding read-only "snapshot" data (e.g., a price at time of purchase) is safe even if the source can change later — see the Computed/Snapshot note in the Common Mistakes section.

Design Pattern 2 — Referencing

The idea: store just a reference (ID) instead of the full data, similar to a Foreign Key.

```
// in users
{ _id: 1, name: "Ashraf" }

// in projects
{ _id: 101, title: "GoRent", ownerId: 1 }
```

When to Use It

- The relationship is a **large one-to-many** or **many-to-many**.
- The related data **changes frequently and independently** of the other side.
- The related data is **shared** across multiple documents (e.g., one product referenced by many orders).
- The array could **grow without bound** (e.g., a very large number of comments).

Pros

- Documents stay small and organized.
- Updating shared data (e.g., a user's profile) only requires updating one place.
- No data duplication.

Cons

- Requires a \$lookup (like a JOIN) at read time — slower than embedding.
- Not atomic — updating two related documents together requires a transaction.

TIPS & TRICKS

- Always index the field you're referencing by (e.g., `ownerId`), since it's effectively your "foreign key lookup" field.
- If you find yourself doing `$lookup` on the same field in almost every query, consider whether a `Subset` or `Computed` pattern could reduce that need.
- Two-way referencing (both sides store each other's IDs) is fine for many-to-many, as long as you pick a clear "primary" side to update first in your application logic.

Design Pattern 3 — Subset Pattern

The idea: instead of fully embedding or fully referencing, use a middle ground — store only a small portion of the related data (e.g., the most recent/most relevant items), while the full data lives in a separate collection with a reference.

```
// Product document holds only the latest 5 reviews, not all of them
{
  _id: 1,
  title: "Laptop HP",
  price: 15000,
  recentReviews: [ // Subset only – latest 5
    { user: "Sara", rating: 5, comment: "Great!" },
    { user: "Omar", rating: 4, comment: "Good" }
  ],
  totalReviewsCount: 230 // pre-computed total
}

// The full set of reviews lives in a separate collection
db.reviews.find({ productId: 1 })
```

When to Use It

When you have a large amount of related data (like reviews), but you also need a small portion of it to appear instantly alongside the parent (e.g., showing the latest 3 ratings on a product page without an extra query).

Practical example: in an e-commerce API, this is the ideal solution for displaying "latest 3 reviews" on a product page without running an aggregation on every page load.

TIPS & TRICKS

- Keep the subset small and purpose-built (e.g., "latest 3–5 items") — it's a display optimization, not a full copy.
- Update the subset only when new data is added to the full collection (e.g., a background job or an application-level trigger), not on every read.
- Combine this pattern with the Computed Pattern: store both a small subset (recentReviews) and a pre-calculated aggregate (averageRating) side by side.

Design Pattern 4 — Bucket Pattern

The idea: instead of creating a separate document for every single reading/event, group a batch of readings from the same time window into one document — commonly used with time-series data.

```
// Instead of one document per second (thousands of documents)
// group all readings for one hour into a single document
{
  sensorId: "sensor_001",
  date: "2026-07-12",
  hour: 14,
  readings: [
    { minute: 0, temp: 25.3 },
    { minute: 1, temp: 25.5 },
    { minute: 2, temp: 25.4 }
    // ... rest of the hour's minutes
  ],
  avgTemp: 25.4,
  maxTemp: 26.1,
  minTemp: 24.8
}
```

When to Use It

- Time-series data (IoT sensor readings, logs, analytics events).
- A massive number of closely-spaced-in-time readings.

Pros

- Dramatically reduces the total number of documents (thousands instead of millions).
- Indexes stay smaller and faster.
- Easy to pre-compute aggregates (avg, max, min) and store them inside the bucket itself.

TIPS & TRICKS

- Pick a bucket window (hour, day) based on your query patterns — how granular do reads actually need to be?
- MongoDB's native Time Series Collections (available since v5.0) automate much of this pattern — worth checking if your MongoDB version supports it before hand-rolling buckets.
- Watch out for buckets that grow unbounded within the time window if the event rate spikes unexpectedly.

Design Pattern 5 — Computed Pattern

The idea: instead of recalculating a value every time it's requested (e.g., averaging all reviews on every page load), calculate it once and store it as a ready-made field, updating it only when the underlying data changes.

```
// Instead of running an aggregation every time the product page loads
{
  _id: 1,
  title: "Laptop HP",
  price: 15000,
  averageRating: 4.5,      // pre-computed
  totalReviews: 230       // pre-computed
}

// When a new review is added, only these two fields get updated –
// not recalculated from all 230 reviews on every page view
```

When to Use It

- A calculation is **read very frequently** (every time a product page opens) but **changes relatively rarely** (a new review every so often).
- You want to avoid an expensive aggregation (e.g., averaging across thousands of documents) on every request.

Practical example: in an e-commerce API, instead of running the aggregation pipeline that computes `avgRating` every time a product page loads, store the result directly on the Product document and update it only when a review is added or removed.

TIPS & TRICKS

- Update computed fields via application logic right after the write that changes them (e.g., inside the same request that creates a review), not through a separate cron job unless near-real-time accuracy isn't required.
- For high write volume, consider using MongoDB transactions to keep the computed field and the source data consistent.
- This pattern trades a small amount of write complexity for a large gain in read performance — worth it wherever reads vastly outnumber writes.

Relationship Types

One-to-One

One entity is linked to exactly one entity of another type.

```
// A user and their detailed profile info (rarely read together)
{ _id: 1, name: "Ashraf", profileDetails: { bio: "...", cv: "..." } }
```

Decision: usually embed, unless the data is very large or rarely read alongside the parent.

One-to-Few

One entity is linked to a small number (2–100) of entities of another type.

```
// A user and their addresses (2–3 addresses)
```

Decision: Embed — the count is small and stable.

One-to-Many

One entity is linked to a relatively large number (hundreds to thousands) of entities of another type.

```
// A category and its products (could be thousands)
```

Decision: Reference, usually — the count is large and constantly changing.

One-to-Squillions

One entity is linked to a massive (millions) number of entities of another type.

```
// A single server and its millions of log entries
```

Decision: Referencing is mandatory — likely combined with the **Bucket Pattern** to reduce document count.

Many-to-Many

Each entity of one type can be linked to multiple entities of the other type, and vice versa.

```
// Students and Courses – each student is enrolled in several courses,  
// and each course has many students  
{ _id: 1, name: "Ashraf", enrolledCourseIds: [101, 102, 103] }  
{ _id: 101, title: "React Basics", enrolledStudentIds: [1, 5, 9] }
```

Decision: usually **two-way referencing**, or referencing from a single direction based on which direction is queried more often.

TIPS & TRICKS

- When in doubt about a relationship's cardinality, estimate the maximum realistic count over the object's lifetime — that number alone often decides between embed and reference.
- For many-to-many, if one direction is queried far more often than the other, consider referencing only in that direction and using a \$lookup for the rare reverse query instead of maintaining both arrays.

The Schema Design Process (Steps 1–4)

Step 1 — Identify the Entities

Write a list of every core "thing" in your system.

Example: User, Product, Order, Review, Category

Step 2 — Identify the Most Common Queries (Access Patterns)

This is **the most important step in NoSQL design**. Before writing any schema, ask yourself:

- What query will run most often? (e.g., "get all products in a given category")
- What actions do users perform frequently? (e.g., "open a product page, view its reviews")
- What's the read-to-write ratio? Is this workload read-heavy or write-heavy?

Write these queries down explicitly before designing anything:

1. Get all products in a category, sorted by price
2. Get a single product with its latest 5 reviews
3. Get all orders for a specific user
4. Search products by name

Step 3 — Identify the Relationships Between Entities

For each pair of entities, determine the type (one-to-one, one-to-few, one-to-many, many-to-many).

User → Orders: One-to-Many
Order → Products: Many-to-Many
Product → Reviews: One-to-Squillions

Step 4 — Apply the Right Pattern to Each Relationship

Ask three questions for every relationship:

1. **Growth rate** — is the count small and stable, or large and increasing?
2. **Update frequency** — does the data change often, or rarely?
3. **Read-together frequency** — is the child data needed every time the parent is fetched?

Based on the answers, choose: Embed / Reference / Subset / Bucket / Computed.

TIPS & TRICKS

- Keep your access-pattern list as a living document — revisit it whenever a new feature changes how data is queried.
- It's fine to mix patterns within the same schema — e.g., embed addresses, reference orders, and use a computed field for ratings, all in the same User/Product design.
- When two access patterns conflict (one favors embedding, another favors referencing), prioritize whichever query runs more frequently or is more performance-critical for the user experience.

Step 5 — Avoiding Common Mistakes

This isn't a "design" step — it's a **review and critique** step. After designing a schema, go back and check it against these common pitfalls.

Mistake 1 — Unbounded Arrays

```
// ❌ Very common mistake
{
  productId: 1,
  reviews: [ /* thousands of reviews embedded directly in the product */ ]
}
```

Problem: this will eventually hit the 16MB limit, and the document becomes slow to read and write well before that limit is even reached.

Fix: use Referencing or the Subset Pattern instead.

Mistake 2 — Deeply Nested Documents

```
// ❌ Common mistake
{
  user: {
    profile: {
      settings: {
        preferences: {
          notifications: {
            email: { enabled: true }
          }
        }
      }
    }
  }
}
```

Problem: deeply nested fields are hard to update, and queries become unwieldy (`user.profile.settings.preferences.notifications.email.enabled`).

Fix: try to keep nesting to 2–3 levels at most.

Mistake 3 — Over-Embedding to "Avoid SQL-Style Joins"

This is the most dangerous mistake — beginners often feel that every embed is a "performance win," so they embed everything even when it doesn't make sense.

```
// ❌ Mistake – embedding data that changes constantly and is shared across
many documents
{
  orderId: 1,
  productDetails: { name: "Laptop", price: 15000, stock: 45 } // full copy of
product data
}
```

Problem: if the product's price changes, you'd need to update every past order containing that product — a disaster.

Fix: the order should store only a **snapshot** of relevant data at the moment of purchase (e.g., the price at sale time), not a live copy that's expected to stay in sync with changes.

Key distinction: embedding a snapshot (immutable, point-in-time data) is safe and often correct. Embedding a live copy of mutable data that's also stored elsewhere is the actual mistake.

TIPS & TRICKS

- Before finalizing any schema, explicitly write "is this field a snapshot or a live copy?" next to every embedded object that duplicates data from elsewhere.
- Run `db.collection.find(...).explain("executionStats")` against your top access-pattern queries to catch accidental COLLSCANS early.
- Periodically check actual document sizes in production (`db.collection.stats()`) — unbounded arrays often creep up slowly before becoming a real problem.

Step 6 — Auditing a Real Project

This is a **real, practical application step** — not a hypothetical exercise. Take an actual schema from a live project you've built (e.g., GoRent or an e-commerce API) and run a full audit on it.

How to Run the Audit

1. Open every schema you have (Product, Order, Review, etc.).
2. For each one, ask: "Why did I choose to embed here? Why did I choose to reference there?" If the honest answer is "I'm not sure, it just felt easier at the time," that decision was made without real analysis.
3. Go back to your project's real access-pattern list — what queries actually run often on the live site?
4. Compare: does the current schema serve those queries efficiently, or is it doing unnecessary \$lookups?

Possible real example: in an e-commerce API, if the **Review** schema fully references both **Product** and **User**, and every fetch runs two \$lookups — it's worth asking whether the Computed Pattern would be a better fit, storing a ready-made **averageRating** directly on the Product instead of recalculating it via aggregation every time.

TIPS & TRICKS

- Turn the audit into a recurring habit — revisit schemas every few months as usage patterns evolve, not just once at project launch.
- Use MongoDB's **\$indexStats** aggregation stage to see which indexes are actually being used in production — drop the ones that aren't.
- Document each schema decision with a one-line comment in code (e.g., **// embedded: 1-to-few, always read together**) so future-you (or teammates) understand the reasoning without re-deriving it.
- When migrating a schema decision (e.g., from embed to reference), plan a data migration script and test it on a copy of production data first.

Topic Ten — Auditing a Real Project

MODULE 3

MONGODB SCHEMA VALIDATION

TECHNICAL REFERENCE

MongoDB Schema Validation

Complete Guide — Rules, Options & a Real-World Hiring Platform
Example

Technical Reference · MongoDB Document Validation

Table of Contents

1. The Core Idea
2. \$jsonSchema — The Recommended Approach
3. validationLevel — How Strictly Rules Apply
4. validationAction — What Happens on Violation
5. Adding Validation to an Existing Collection
6. Real-World Example — Training & Hiring Platform
7. Bypassing Validation (bypassDocumentValidation)
8. Inspecting & Testing Validation Rules
9. Validation vs. Application-Level Validation
10. Important Notes & Limitations

The Core Idea

Since NoSQL databases are flexible by nature, that flexibility is both a strength and a risk — anyone can insert a document in almost any shape, even one missing essential fields or containing the wrong data types. **Schema Validation** is MongoDB's built-in mechanism for enforcing rules at the collection level, without fully sacrificing NoSQL's flexibility.

Rules are attached to a collection, and any document being inserted or updated must comply with them, or MongoDB rejects the operation.

```
db.createCollection("users", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["name", "email", "age"],
      properties: {
        name: {
          bsonType: "string",
          description: "must be a string and is required"
        },
        email: {
          bsonType: "string",
          pattern: "^.+@.+$",
          description: "must be a valid email format"
        },
        age: {
          bsonType: "int",
          minimum: 18,
          maximum: 100,
          description: "must be an integer between 18 and 100"
        }
      }
    }
  }
})
```

Attempting this insert:

```
db.users.insertOne({ name: "Ashraf", age: 15 })
```

...would be rejected for two reasons: [email](#) is missing (required), and [age](#) is below the minimum (18).

TIP

- Validation is enforced at the database engine level — it applies regardless of which application, script, or admin tool is writing the data.
- You can add validation to a brand-new collection (via [createCollection](#)) or retrofit it onto an existing one (via [collMod](#)), covered in Section Five.

\$jsonSchema — The Recommended Approach

This is the official standard (JSON Schema) that MongoDB supports for validation. It's more expressive and portable than MongoDB's older query-operator-based validators.

Key Properties

Property	Purpose
<code>bsonType</code>	The data type (string, int, double, array, object, date, objectId, etc.)
<code>required</code>	An array of field names that must be present
<code>properties</code>	Defines each field and its constraints
<code>minimum</code> / <code>maximum</code>	Numeric bounds
<code>minLength</code> / <code>maxLength</code>	String length bounds
<code>pattern</code>	Regex used to validate string format
<code>enum</code>	A fixed list of allowed values
<code>additionalProperties</code>	Whether undefined extra fields are allowed (true/false)
<code>items</code>	Validation rules applied to each element of an array field
<code>minItems</code> / <code>maxItems</code>	Bounds on array length

A Fuller Example

```
db.createCollection("products", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["title", "price", "category"],
      additionalProperties: false,
      properties: {
        _id: {},
        title: { bsonType: "string", minLength: 3, maxLength: 100 },
        price: { bsonType: "double", minimum: 0 },
        category: {
          enum: ["Electronics", "Clothing", "Books", "Food"]
        },
        stock: { bsonType: "int", minimum: 0 },
        tags: {
          bsonType: "array",
          items: { bsonType: "string" },
          maxItems: 10
        }
      }
    }
  }
})
```

Note: setting `additionalProperties: false` is strict — any field not explicitly declared in `properties` will cause the document to be rejected. Use it carefully on collections where the schema is expected to evolve frequently.

validationLevel — How Strictly Rules Apply

Determines whether the rule is enforced on everything, or only on new writes.

```
db.runCommand({
  collMod: "products",
  validationLevel: "strict"  // or "moderate"
})
```

Value	Meaning
<code>strict</code> (default)	Applied to all inserts and updates, even on existing documents that don't already comply with the rule
<code>moderate</code>	Applied to new inserts, but allows updates to existing non-compliant documents as long as the update doesn't make them worse

Practical use: if an existing collection has legacy data that doesn't match a newly added rule, use `moderate` so you don't break anything currently working, then clean up the old data gradually.

validationAction — What Happens on Violation

```
db.runCommand({
  collMod: "products",
  validationAction: "warn"  // or "error"
})
```

Value	Meaning
error (default)	Rejects the operation entirely
warn	Allows the operation but logs a warning — useful during initial testing before enforcing the rule strictly

TIP

- A common rollout strategy: deploy new validation rules with `validationAction: "warn"` first, monitor the logs for a few days to see how many existing writes would have failed, then switch to `"error"` once you're confident nothing legitimate gets blocked.

Adding Validation to an Existing Collection

If the collection already exists and you need to add rules to it:

```
db.runCommand({
  collMod: "users",
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["email"],
      properties: {
        email: { bsonType: "string" }
      }
    }
  },
  validationLevel: "moderate"
})
```

This is the safer path for production collections that already contain live data — combining `collMod` with `validationLevel: "moderate"` avoids suddenly breaking writes to older, non-compliant documents.

Real-World Example — Training & Hiring Platform

Continuing the same platform scenario used in the aggregation examples: a training and hiring platform with trainees, mentors, and admins, each carrying different required fields.

```
db.createCollection("users", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["name", "email", "role", "joinDate"],
      properties: {
        name: { bsonType: "string", minLength: 2, maxLength: 60 },
        email: {
          bsonType: "string",
          pattern: "^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$",
          description: "must be a valid email address"
        },
        role: {
          enum: ["trainee", "mentor", "admin"],
          description: "must be one of the three defined roles"
        },
        age: {
          bsonType: "int",
          minimum: 16,
          maximum: 70
        },
        city: { bsonType: "string" },
        track: {
          bsonType: ["string", "null"],
          description: "trainees/mentors have a track; admins have null"
        },
        isActive: { bsonType: "bool" },
        joinDate: { bsonType: "date" },
        completedCourses: {
          bsonType: "int",
          minimum: 0
        },
        monthlyIncome: {
          bsonType: ["int", "double"],
          minimum: 0
        },
        skills: {
          bsonType: "array",
          items: { bsonType: "string" },
          maxItems: 30
        }
      }
    }
  }
})
```

```

        appliedJobsCount: {
          bsonType: "int",
          minimum: 0
        }
      }
    },
    validationLevel: "strict",
    validationAction: "error"
  })

```

Notice `bsonType: ["string", "null"]` on the `track` field — this allows the field to be either a string or null, matching the earlier scenario where an admin user has no track at all.

What This Prevents in Practice

Attempted Insert	Why It's Rejected
<code>{ name: "Test", email: "not-an-email", role: "trainee" }</code>	Fails the <code>email</code> pattern check, and is missing the required <code>joinDate</code>
<code>{ name: "A", email: "a@b.com", role: "supervisor", joinDate: new Date() }</code>	<code>role</code> isn't one of the allowed enum values
<code>{ name: "Karim", email: "k@x.com", role: "trainee", age: 12, joinDate: new Date() }</code>	<code>age</code> is below the minimum of 16

Rolling Out Safely on a Live Collection

If the `users` collection already has thousands of live records when this rule is introduced, apply it gradually:

```

db.runCommand({
  collMod: "users",
  validator: { $jsonSchema: { /* same schema as above */ } },
  validationLevel: "moderate",
  validationAction: "warn"
})

```

Then monitor the MongoDB logs for validation warnings over a few days, fix any legacy records that would fail, and only then switch to `validationLevel: "strict"` and `validationAction: "error"` for full enforcement.

Bypassing Validation (bypassDocumentValidation)

Sometimes you need to intentionally skip validation — for example, during a one-off data migration or bulk import where you know the format temporarily differs from the current rules.

```
db.users.insertMany(  
  [ /* legacy documents */ ],  
  { bypassDocumentValidation: true }  
)
```

Caution: this flag requires specific privileges in a production MongoDB deployment (typically restricted to admin-level roles), and should be used deliberately, not as a routine workaround for failing validation.

Inspecting & Testing Validation Rules

Viewing the Current Validator

```
db.getCollectionInfos({ name: "users" })
```

This returns the validator currently applied to the collection, along with its `validationLevel` and `validationAction`.

Testing Before Enforcing

A practical workflow for introducing validation on a collection already in production:

1. Add the validator with `validationAction: "warn"` so nothing breaks immediately.
2. Run typical application traffic for a few days and check the logs for validation warnings.
3. Identify and fix (or migrate) documents that would fail under the new rule.
4. Switch to `validationAction: "error"` once confident the rule won't block legitimate operations.

Validation vs. Application-Level Validation

	MongoDB Schema Validation	Application-Level (Mongoose/Joi)
Where it runs	Inside the database engine itself	Inside your Node.js/Express backend
Coverage	Protects against <i>any</i> write, including direct DB access, scripts, and migrations	Only protects writes that go through your application code
Error messages	Generic MongoDB validation errors	Can be customized with detailed, user-friendly messages
Business logic	Not suited for complex conditional rules	Well suited for rich business logic and cross-field conditional checks

Best practice: use both together. Application-level validation (e.g., Mongoose schemas) gives fast, user-friendly feedback during normal app usage. MongoDB schema validation acts as a last line of defense at the database layer, catching anything that bypasses the application — a rogue script, a direct database console session, or a bug in the backend logic.

Important Notes & Limitations

- **Not a replacement for backend validation** (e.g., Mongoose schemas or Joi) — it's an additional layer of protection at the database level, not a substitute. If someone accesses the database directly (an admin, a script, a migration), database-level validation still protects data integrity even if the backend was bypassed.
- **Performance** — validation adds a small overhead to every insert/update, but this is usually negligible compared to the benefit of guaranteed data integrity.
- **Not meant for complex business logic** — rules like "this product can only be sold if there's an active promotional offer" are application logic, not something that belongs in schema validation.
- **Schema evolution** — as your application changes, remember to update the validator too; a forgotten stale validator can silently block legitimate new fields.
- **No cross-document rules** — `$jsonSchema` validates a document against itself; it cannot enforce rules that depend on data in other documents or collections (e.g., "this email must be unique across the whole collection" — that still requires a unique index).

TIPS & TRICKS

- Combine schema validation with a `unique` index where needed — validation checks shape and type, indexes enforce uniqueness across documents.
- Keep a copy of every collection's validator schema in your project's codebase (e.g., a `/schemas` folder) so it's version-controlled alongside your Mongoose models, not just living invisibly inside the database.
- When adding a new required field to a schema that already has production data, always roll out with `moderate` + `warn` first — treat it the same as a database migration, not a simple config change.

MODULE 4

MONGOOSE IN DEPTH

TECHNICAL REFERENCE

Mongoose In Depth

Schemas, Instance & Static Methods, Virtuals, Middleware & Population

Technical Reference · Mongoose / Node.js / MongoDB

Table of Contents

1. Schema — Beyond Just Defining Fields
2. Important Schema Options Often Overlooked
3. Instance Methods vs. Static Methods vs. Virtuals
4. Middleware (Hooks) — Pre and Post
5. The "this" Context Trap in Middleware
6. Population — Replacing Manual Joins
7. The N+1 Query Problem & When to Avoid Populate
8. Mongoose Validation vs. MongoDB Schema Validation
9. Custom Error Messages with Enum
10. Indexes in Mongoose
11. Summary — What to Use When

Schema — Beyond Just Defining Fields

Basic Definition

```
const mongoose = require("mongoose");

const productSchema = new mongoose.Schema({
  title: {
    type: String,
    required: [true, "Product title is required"],
    trim: true,
    minlength: 3,
    maxlength: 100
  },
  price: {
    type: Number,
    required: true,
    min: 0
  },
  category: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "Category",
    required: true
  }
}, {
  timestamps: true // automatically adds createdAt and updatedAt
});
```

Important Schema Options Often Overlooked

```
const productSchema = new mongoose.Schema({
  // ... fields
}, {
  timestamps: true,

  toJSON: { virtuals: true },    // makes virtuals appear when converted to
  // JSON
  toObject: { virtuals: true },

  strict: true,    // (default: true) – any field not defined in the schema
  // is ignored automatically

  versionKey: false, // removes the __v field Mongoose adds automatically for
  // version tracking

  collection: "products" // manually set the collection name (if different
  // from default)
});
```

Important note on "strict": setting it to **false** means any field on the document not defined in the schema gets saved anyway — this defeats one of the schema's most valuable protections (guarding against unexpected data). Keep it **true** unless you have a strong reason not to.

Instance Methods vs. Static Methods vs. Virtuals

Instance Methods — Operate on a Single Document

```
productSchema.methods.isInStock = function() {  
  return this.quantity > 0;  
};  
  
// Usage:  
const product = await Product.findById(id);  
if (product.isInStock()) { ... }
```

Static Methods — Operate on the Model Itself

```
productSchema.statics.findByCategory = function(categoryId) {  
  return this.find({ category: categoryId });  
};  
  
// Usage:  
const products = await Product.findByCategory(categoryId);
```

Virtuals — Computed Fields, Not Actually Stored

```
productSchema.virtual("priceAfterDiscount").get(function() {  
  return this.price - (this.price * this.discount / 100);  
});  
  
// Or a virtual that pulls data from another collection (reverse population)  
productSchema.virtual("reviews", {  
  ref: "Review",  
  foreignField: "product",  
  localField: "_id"  
});
```

Important: a virtual is never stored in the database — it's calculated only when accessed. You must enable `toJSON: { virtuals: true }` for it to show up in responses.

Concept	Use It When...
Instance Methods	You need an operation tied to a specific document (e.g., <code>comparePassword</code>)
Static Methods	You need a custom query that repeats often (e.g., <code>findActiveUsers</code>)
Virtuals	You need a computed field that shouldn't be stored (e.g., <code>fullName</code> , <code>priceAfterDiscount</code>)

Middleware (Hooks) — Pre and Post

Middleware functions run **automatically before or after** a given operation (save, find, remove, etc.).

Type	When It Runs
pre	Before the operation
post	After the operation

Example 1 — pre("save"): Hashing the Password Before Saving

```
const bcrypt = require("bcryptjs");

userSchema.pre("save", async function(next) {
  if (!this.isModified("password")) return next(); // skip re-hashing if
  password wasn't changed
  this.password = await bcrypt.hash(this.password, 12);
  next();
});
```

A common beginner question: why `this.isModified("password")`? Because if you call `save()` to update a different field (like the name), this check prevents the password from being re-hashed every single time you save, even though it's already hashed from before.

Example 2 — pre("find"): Hiding Soft-Deleted Products

```
productSchema.pre(/^find/, function(next) {
  this.find({ isDeleted: { $ne: true } });
  next();
});
```

`/^find/` is a regex that covers every operation starting with "find" (`find`, `findOne`, `findById`, etc.) at once.

Example 3 — `post("save")`: Logging or Sending an Email

```
orderSchema.post("save", function(doc) {  
  console.log(`Order ${doc._id} was created`);  
  // could send a confirmation email here, for example  
});
```

Example 4 — `pre("findOneAndDelete")`: Cascade Delete

```
// If a Category is deleted, delete all products linked to it  
categorySchema.pre("findOneAndDelete", async function(next) {  
  const categoryId = this.getQuery()._id;  
  await Product.deleteMany({ category: categoryId });  
  next();  
});
```

The "this" Context Trap in Middleware

A point that confuses many beginners: what `this` refers to changes depending on the middleware type.

```
// In document middleware (save, validate) – "this" refers to the Document
userSchema.pre("save", function(next) {
  console.log(this.name); // ✅ works
  next();
});

// In query middleware (find, findOne, updateOne) – "this" refers to the Query
// itself, not a Document
productSchema.pre("find", function(next) {
  console.log(this.getQuery()); // ✅ works – returns the search condition
  console.log(this.name);      // ❌ undefined – there's no document yet
  next();
});
```

TIP

- Never use arrow functions for middleware callbacks — arrow functions don't bind their own `this`, so it will point to the surrounding scope instead of the document/query.
- When debugging middleware behavior, log `this.constructor.modelName` to confirm which model context you're actually inside.

Population — Replacing Manual Joins

The Core Idea

Once you define a [ref](#) in the schema, you can replace an ObjectId with the full referenced document at query time.

```
const orderSchema = new mongoose.Schema({
  user: { type: mongoose.Schema.Types.ObjectId, ref: "User" },
  products: [{ type: mongoose.Schema.Types.ObjectId, ref: "Product" }]
});
```

```
// Without populate
const order = await Order.findById(id);
console.log(order.user); // just the ObjectId

// With populate
const order = await Order.findById(id).populate("user");
console.log(order.user); // the full User document
```

Populate with Field Selection (Like Projection)

```
const order = await Order.findById(id).populate("user", "name email");
// only fetches name and email from User, not everything
```

Multiple Populates Together

```
const order = await Order.findById(id)
  .populate("user", "name email")
  .populate("products", "title price");
```

Nested Population — When a Reference Itself Has Another Reference

```
const review = await Review.findById(id).populate({
  path: "product",
  populate: {
    path: "category",
    select: "name"
  }
});
```

Populate with Filter and Sort

```
const category = await Category.findById(id).populate({
  path: "products",
  match: { price: { $gte: 100 } },
  select: "title price",
  options: { sort: { price: -1 }, limit: 5 }
});
```

The N+1 Query Problem & When to Avoid Populate

Populate performs a **separate additional query** for each reference — it's not a true JOIN like in SQL. If you have an array of 100 orders and each one is populated, this can result in hundreds of queries.

The N+1 problem: one query to get the list, then N additional queries (one per referenced document) to populate each one — performance degrades quickly as the list grows.

The Better Alternative for High Performance

Use the Aggregation Pipeline with `$lookup` instead of Populate, especially with large datasets or a high query volume.

```
// Instead of Populate, you can use:
Order.aggregate([
  { $match: { _id: orderId } },
  { $lookup: { from: "users", localField: "user", foreignField: "_id", as:
"user" } }
])
```

TIPS & TRICKS

- Populate is fine for typical CRUD endpoints with reasonable list sizes and pagination — don't over-optimize prematurely.
- Reach for `$lookup`-based aggregation when a single request would otherwise trigger dozens or hundreds of populate queries, or when you need to combine population with grouping/filtering logic in the same query.
- Always paginate lists that use populate — an unbounded list with nested populate is a common source of slow endpoints in production.

Mongoose Validation vs. MongoDB Schema Validation

```
const userSchema = new mongoose.Schema({
  email: {
    type: String,
    required: [true, "Email is required"],
    unique: true,
    lowercase: true,
    validate: {
      validator: function(value) {
        return /^\\S+@\\S+\\.\\S+$/\\.test(value);
      },
      message: "Invalid email format"
    }
  },
  age: {
    type: Number,
    min: [18, "Must be at least 18 years old"],
    max: [100, "Age cannot exceed 100"]
  }
});
```

The difference from MongoDB's own \$jsonSchema validation: Mongoose validation runs at the application layer (Node.js), before the data even reaches the database. It responds to the user faster, but doesn't protect you if someone accesses the database directly, bypassing your application entirely.

Custom Error Messages with Enum

```
const orderSchema = new mongoose.Schema({
  status: {
    type: String,
    enum: {
      values: ["pending", "confirmed", "shipped", "delivered"],
      message: "{VALUE} is not a valid status"
    },
    default: "pending"
  }
});
```

The `{VALUE}` placeholder is automatically replaced with the actual invalid value submitted, producing a clear, specific error message.

Indexes in Mongoose

```
userSchema.index({ email: 1 }, { unique: true });  
productSchema.index({ category: 1, price: -1 });
```

These map directly to the same index concepts covered earlier at the MongoDB shell level — Mongoose simply lets you declare them alongside the schema definition itself, keeping index intent version-controlled with your model code.

TIP

- Mongoose builds indexes automatically when the app starts (in development), which can be slow on large collections — in production, create indexes explicitly via migration scripts or the shell instead of relying on auto-indexing.

Summary — What to Use When

Concept	Use It When...
Instance Methods	You need an operation tied to a specific document (e.g., <code>comparePassword</code>)
Static Methods	You need a custom query that repeats often (e.g., <code>findActiveUsers</code>)
Virtuals	You need a computed field that shouldn't be stored (e.g., <code>fullName</code> , <code>priceAfterDiscount</code>)
<code>pre("save")</code>	Modifying data before it's saved (hashing, formatting)
<code>pre("find")</code>	Automatic filtering (e.g., soft delete)
<code>post("save")</code>	Something after saving (logging, notifications)
Populate	Query volume is small and manageable
Aggregation + <code>\$lookup</code>	Performance and large data volume matter more than code simplicity

MODULE 5

MONGODB TRANSACTIONS

TECHNICAL REFERENCE

MongoDB Transactions

Multi-Document ACID Guarantees — Concepts, Mechanics & Real-World
Usage

Technical Reference · MongoDB / Mongoose

Table of Contents

1. Why Transactions Are Needed
2. How Transactions Solve the Problem (ACID Guarantee)
3. The Core Steps of Any Transaction
4. Requirements to Run Transactions
5. Write Conflicts & Retry Logic
6. Read Concern & Write Concern Inside Transactions
7. When to Use Transactions (and When Not To)
8. Real-World Example — Cash Order Checkout
9. The Performance Cost of Transactions

Why Transactions Are Needed

In NoSQL, a single document is inherently **atomic** — updating one document is guaranteed to happen entirely or not at all. The problem appears when an operation needs to update **more than one document across more than one collection** at the same time.

The Classic Problem Example

Consider a "purchase a product" operation:

1. Decrease stock quantity in `products`
2. Create a new order in `orders`
3. Deduct funds from the user's balance in `users`

```
// ❌ Without a transaction – a real risk
await Product.updateOne({ _id: productId }, { $inc: { stock: -1 } });
// The server crashes right here!
await Order.create({ userId, productId, status: "confirmed" });
await User.updateOne({ _id: userId }, { $inc: { balance: -price } });
```

If the server crashes right after the first step, the result is: **stock was decremented, but no order was ever created, and no money was deducted** — a product vanished for no reason, and the customer never paid. This is exactly what transactions prevent.

How Transactions Solve the Problem (ACID Guarantee)

A transaction guarantees that all these operations execute as **a single unit (all or nothing)**:

- If everything succeeds → **commit** (all changes are persisted together)
- If anything fails at any step → **abort** (everything is rolled back, including steps that had already succeeded)

```
const session = await mongoose.startSession();
session.startTransaction();

try {
  await Product.updateOne(
    { _id: productId },
    { $inc: { stock: -1 } },
    { session }
  );

  await Order.create(
    [{ userId, productId, status: "confirmed" }],
    { session }
  );

  await User.updateOne(
    { _id: userId },
    { $inc: { balance: -price } },
    { session }
  );

  await session.commitTransaction();
  console.log("Transaction succeeded");
} catch (error) {
  await session.abortTransaction();
  console.log("Transaction failed, everything rolled back:", error);
} finally {
  session.endSession();
}
```

Important: every operation must pass the same `session` as an option, or it won't be treated as part of the same transaction.

The Core Steps of Any Transaction

Step	Command	Purpose
1	<code>startSession()</code>	Begins a new session tied to the transaction
2	<code>startTransaction()</code>	Actually starts the transaction
3	Run operations passing { <code>session</code> }	Each operation is recorded as part of the transaction, not applied to real data immediately
4	<code>commitTransaction()</code>	Persists all changes at once
5	<code>abortTransaction()</code> (on error)	Rolls back everything
6	<code>endSession()</code>	Closes the session and frees resources

Requirements to Run Transactions

Critical: transactions are **not available in every MongoDB setup**.

- MongoDB must be running as a **Replica Set** (even a single-node replica set for local development) or a **Sharded Cluster**.
- **Not available on a standalone MongoDB instance** (a plain server with no replica set).
- Supported from **MongoDB 4.0** onward for replica sets, and from **4.2** onward for sharded clusters.

```
// For local development, run a single-node replica set:  
mongod --replSet rs0  
// then initiate it:  
rs.initiate()
```

Attempting to use a transaction on a standalone instance results in a clear error stating the feature isn't supported.

Write Conflicts & Retry Logic

If another transaction tries to modify the same document at the same time, MongoDB throws a **Write Conflict Error**. This is part of the **Isolation** guarantee in ACID.

```
try {
  // the operation
} catch (error) {
  if (error.errorLabels &&
error.errorLabels.includes("TransientTransactionError")) {
    // safe to retry – this error is temporary
  }
  await session.abortTransaction();
}
```

BEST PRACTICE

- Your code should be prepared to **retry** when a **TransientTransactionError** occurs, since it indicates a temporary conflict between transactions, not an actual logic error.
- MongoDB drivers provide helper wrappers (e.g., **withTransaction()** in the native driver) that automatically handle retry logic for transient errors — worth using instead of hand-rolling retry loops.

Read Concern & Write Concern Inside Transactions

You can control the accuracy-vs-speed trade-off explicitly:

```
session.startTransaction({
  readConcern: { level: "snapshot" },    // read a fixed snapshot of data as
  of transaction start
  writeConcern: { w: "majority" }        // writes must be acknowledged by a
  majority of nodes before success
});
```

`readConcern: "snapshot"` matters a lot because it guarantees you're reading the **same version of the data** throughout the transaction's lifetime, even if someone else modifies the data at the same moment.

When to Use Transactions (and When Not To)

Use Them When:

- An operation touches **more than one collection** and all parts must succeed or fail together (like the purchase example above).
- Financial operations (transferring balance, deducting/adding credit).
- Operations with a serious **race condition** risk (e.g., booking the last available ticket or the last item in stock).

Avoid Them When:

- The operation touches **a single document only** — MongoDB documents are already atomic, so an extra transaction adds nothing.
- You have very high throughput (many operations per second) — transactions add noticeable performance overhead.
- The problem can be solved with a better schema design instead (e.g., embedding) rather than needing coordination across separate collections in the first place.

Real-World Example — Cash Order Checkout

In an e-commerce API, the **Cash Order** scenario is one of the best real-world fits for a transaction.

```

const createCashOrder = async (req, res) => {
  const session = await mongoose.startSession();
  session.startTransaction();

  try {
    const cart = await Cart.findById(req.params.cartId).session(session);

    const order = await Order.create(
      [{
        user: req.user._id,
        cartItems: cart.cartItems,
        totalOrderPrice: cart.totalPriceAfterDiscount || cart.totalPrice,
        paymentMethodType: "cash"
      }],
      { session }
    );

    const bulkOptions = cart.cartItems.map((item) => ({
      updateOne: {
        filter: { _id: item.product, quantity: { $gte: item.quantity } },
        update: { $inc: { quantity: -item.quantity, sold: +item.quantity } }
      }
    }));

    const bulkResult = await Product.bulkWrite(bulkOptions, { session });

    // If any product didn't have enough stock, bulkWrite won't update it at
    all
    if (bulkResult.modifiedCount !== cart.cartItems.length) {
      throw new Error("Insufficient stock for one or more products");
    }

    await Cart.findByIdAndDelete(req.params.cartId).session(session);

    await session.commitTransaction();
    res.status(201).json({ status: "success", data: order });
  } catch (error) {
    await session.abortTransaction();
    res.status(400).json({ status: "fail", message: error.message });
  } finally {
    session.endSession();
  }
}

```

```
}  
};
```

Why this matters here specifically: if any error occurs after the order was already created (e.g., a stock update failure), without a transaction the customer could end up with a confirmed order while the product was never actually deducted from stock — a genuine data inconsistency.

The Performance Cost of Transactions

Transactions aren't "free" — they take longer than regular operations and hold resources on the server until they're closed (commit/abort). Using them for every simple operation without real need will negatively affect overall system performance.

GOLDEN RULE

- Only use a transaction when you genuinely need an "all or nothing" guarantee across more than one document/collection — not as a default solution for every update.
- Keep transactions short-lived: avoid slow operations (like external API calls or heavy computation) inside a transaction block, since long-running transactions hold locks longer and increase the chance of write conflicts with other operations.
- For single-document updates, rely on MongoDB's built-in document-level atomicity — no transaction wrapper needed.

MODULE 6

MONGODB AGGREGATION USERS EXAMPLE

APPLIED EXAMPLE

MongoDB Aggregation Pipeline

Real-World "Users" Collection — Training & Hiring Platform Scenario

Technical Reference · Applied Aggregation & Indexing

Table of Contents

1. The Scenario & Seed Data
2. \$match — Filter Active Trainees by Track
3. \$group — Employment Rate per Track
4. \$sort + \$limit — Top Performing Trainees
5. \$project — Hiring-Ready Report
6. \$addFields — Months Since Joining
7. \$count — Dropped-Out Trainees
8. \$unwind — Most Common Skills
9. \$lookup — Matching Job Openings
10. \$facet — Full Dashboard in One Query
11. \$bucket — Progress Level Groups
12. \$merge — Monthly Snapshot Reports
13. Indexes on the Users Collection

SECTION ONE

The Scenario & Seed Data

Imagine building a **tech training & hiring platform** (similar to the DEY HireReady Initiative). The `users` collection holds trainees, mentors, and admins.

```
db.users.insertMany([
  {
    _id: 1, name: "Ahmed Sayed", email: "ahmed@gmail.com", age: 24,
    city: "Cairo", track: "MERN Stack", role: "trainee", isActive: true,
    joinDate: new Date("2025-01-15"), completedCourses: 8, monthlyIncome: 0,
    skills: ["React", "Node.js", "MongoDB"], appliedJobsCount: 15
  },
  {
    _id: 2, name: "Mona Khalil", email: "mona@gmail.com", age: 27,
    city: "Alexandria", track: ".NET", role: "trainee", isActive: true,
    joinDate: new Date("2025-02-01"), completedCourses: 12, monthlyIncome:
8000,
    skills: ["C#", "ASP.NET Core", "SQL Server"], appliedJobsCount: 20
  },
  {
    _id: 3, name: "Youssef Adel", email: "youssef@gmail.com", age: 23,
    city: "Cairo", track: "MERN Stack", role: "trainee", isActive: false,
    joinDate: new Date("2024-11-10"), completedCourses: 3, monthlyIncome: 0,
    skills: ["JavaScript"], appliedJobsCount: 2
  },
  {
    _id: 4, name: "Salma Reda", email: "salma@gmail.com", age: 30,
    city: "Menoufia", track: "Instructor", role: "mentor", isActive: true,
    joinDate: new Date("2023-06-01"), completedCourses: 0, monthlyIncome:
15000,
    skills: ["Teaching", "MERN Stack", "Mentorship"], appliedJobsCount: 0
  },
  {
    _id: 5, name: "Karim Fathy", email: "karim@gmail.com", age: 25,
    city: "Menoufia", track: "MERN Stack", role: "trainee", isActive: true,
    joinDate: new Date("2025-03-20"), completedCourses: 10, monthlyIncome:
6500,
    skills: ["React", "Express", "MongoDB", "Redis"], appliedJobsCount: 25
  },
  {
    _id: 6, name: "Nourhan Adly", email: "nourhan@gmail.com", age: 22,
    city: "Alexandria", track: ".NET", role: "trainee", isActive: true,
    joinDate: new Date("2025-01-05"), completedCourses: 9, monthlyIncome: 0,
    skills: ["C#", "Entity Framework"], appliedJobsCount: 18
  },
  {
```

```
_id: 7, name: "Hossam Admin", email: "hossam@platform.com", age: 33,  
city: "Cairo", track: null, role: "admin", isActive: true,  
joinDate: new Date("2022-01-01"), completedCourses: 0, monthlyIncome:  
20000,  
skills: ["Management"], appliedJobsCount: 0  
}  
])
```

Three types of users exist: **trainee**, **mentor**, and **admin**. Each has a **track**, a **monthlyIncome** (once employed), and an **appliedJobsCount**.

\$match — Filter Active Trainees by Track

```
db.users.aggregate([
  { $match: { role: "trainee", track: "MERN Stack", isActive: true } }
])
```

Result: Ahmed Sayed and Karim Fathy (Youssef is excluded since `isActive: false`).

Real question answered: "I need the list of active trainees in a specific track so I can send them a new job opportunity."

\$group — Employment Rate per Track

```
db.users.aggregate([
  { $match: { role: "trainee" } },
  { $group: {
    _id: "$track",
    totalTrainees: { $sum: 1 },
    employed: { $sum: { $cond: [{ $gt: ["$monthlyIncome", 0] }, 1, 0] } },
    avgAppliedJobs: { $avg: "$appliedJobsCount" }
  }}
])
```

Result:

```
{ "_id": "MERN Stack", "totalTrainees": 3, "employed": 1, "avgAppliedJobs": 14
}
{ "_id": ".NET",      "totalTrainees": 2, "employed": 1, "avgAppliedJobs": 19
}
```

Real question answered: "Which track leads to more employed graduates — MERN or .NET?" — a question management would genuinely ask.

\$sort + \$limit — Top Performing Trainees

```
db.users.aggregate([
  { $match: { role: "trainee" } },
  { $sort: { completedCourses: -1 } },
  { $limit: 3 }
])
```

Result: Mona (12) → Karim (10) → Nourhan (9)

Real question answered: "Who are the top 3 trainees I can recommend to a partner company that's hiring?"

\$project — Hiring-Ready Report

```
db.users.aggregate([
  { $match: { role: "trainee", isActive: true } },
  { $project: {
    _id: 0,
    name: 1,
    track: 1,
    skills: 1,
    readyForHire: { $gte: ["$completedCourses", 8] }
  }}
])
```

Result: A trimmed report — no email or salary (sensitive data) — with a new computed field indicating whether the trainee is "ready for hire."

\$addFields — Months Since Joining

```
db.users.aggregate([
  { $addFields: {
    monthsSinceJoining: {
      $dateDiff: { startDate: "$joinDate", endDate: new Date(), unit:
"month" }
    }
  }}
])
```

Real question answered: "How many months has this trainee been enrolled without finding a job yet? — a strong indicator they need closer follow-up."

\$count — Dropped-Out Trainees

```
db.users.aggregate([
  { $match: { role: "trainee", isActive: false } },
  { $count: "droppedOutTrainees" }
])
```

Result: { "droppedOutTrainees": 1 } (Youssef only).

\$unwind — Most Common Skills

```
db.users.aggregate([
  { $unwind: "$skills" },
  { $group: { _id: "$skills", count: { $sum: 1 } } },
  { $sort: { count: -1 } }
])
```

Result:

```
{ "_id": "MongoDB", "count": 2 }
{ "_id": "React",   "count": 2 }
{ "_id": "C#",      "count": 2 }
...
```

Real question answered: "Which skill is most common among trainees, so I know what supplementary course to build?"

\$lookup — Matching Job Openings

```
db.jobs.insertMany([
  { _id: 101, title: "Junior MERN Developer", requiredTrack: "MERN Stack",
    city: "Cairo" },
  { _id: 102, title: "Junior .NET Developer", requiredTrack: ".NET", city:
    "Alexandria" }
])

db.users.aggregate([
  { $match: { role: "trainee", isActive: true } },
  { $lookup: {
    from: "jobs",
    localField: "track",
    foreignField: "requiredTrack",
    as: "matchingJobs"
  }}
])
```

Result: Each trainee now carries a `matchingJobs` array with openings that match their track — effectively a real "Recommended Jobs" feature.

\$facet — Full Dashboard in One Query

```
db.users.aggregate([
  { $facet: {
    "usersByRole": [
      { $group: { _id: "$role", count: { $sum: 1 } } }
    ],
    "employmentStats": [
      { $match: { role: "trainee" } },
      { $group: {
        _id: null,
        employed: { $sum: { $cond: [{ $gt: ["$monthlyIncome", 0] }, 1, 0]
      } },
        unemployed: { $sum: { $cond: [{ $eq: ["$monthlyIncome", 0] }, 1,
0] } }
      } }
    ],
    "topCity": [
      { $group: { _id: "$city", count: { $sum: 1 } } },
      { $sort: { count: -1 } },
      { $limit: 1 }
    ]
  } }
])
```

Real question answered: "I need one dashboard showing: user distribution by role, employment rate, and the city with the most trainees" — all in a single query instead of three.

\$bucket — Progress Level Groups

```
db.users.aggregate([
  { $match: { role: "trainee" } },
  { $bucket: {
    groupBy: "$completedCourses",
    boundaries: [0, 5, 10, 15],
    default: "Advanced",
    output: { count: { $sum: 1 }, names: { $push: "$name" } }
  }}
])
```

Result:

```
{ "_id": 0, "count": 1, "names": ["Youssef Adel"] }           // Beginner
(0-4 courses)
{ "_id": 5, "count": 3, "names": ["Ahmed", "Nourhan", "Karim"] } //
Intermediate (5-9)
{ "_id": 10, "count": 1, "names": ["Mona Khalil"] }          // Advanced
(10-14)
```


\$merge — Monthly Snapshot Reports

```
db.users.aggregate([
  { $match: { role: "trainee" } },
  { $group: {
    _id: "$track",
    totalActive: { $sum: { $cond: ["$isActive", 1, 0] } },
    employmentRate: { $avg: { $cond: [{ $gt: ["$monthlyIncome", 0] }, 1, 0] } }
  } },
  { $merge: { into: "monthlyReports", whenMatched: "replace" } }
])
```

Real scenario: run this pipeline every month to persist a snapshot into a `monthlyReports` collection, tracking how the employment rate evolves over time.

Indexes on the Users Collection

Unique Index — Prevent Duplicate Registration

```
db.users.createIndex({ email: 1 }, { unique: true })
```

If someone tries registering with an email already on the platform, the database itself rejects the operation.

Compound Index — The Most Common Admin Filter

```
db.users.createIndex({ role: 1, track: 1, isActive: 1 })
```

The most frequent query in the admin dashboard is "get all active trainees in a given track." This index makes it instant even with thousands of users.

```
// Query that benefits from this index
db.users.find({ role: "trainee", track: "MERN Stack", isActive: true })
```

Sparse Index — Since Admins & Mentors Have No Track

```
db.users.createIndex({ track: 1 }, { sparse: true })
```

Hossam Admin has `track: null`. A sparse index skips users without a track instead of indexing them all under a repeated null value.

Partial Index — Only on Inactive Trainees

```
db.users.createIndex(
  { joinDate: 1 },
  { partialFilterExpression: { isActive: false } }
)
```

A recurring "trainees who went inactive" report filtered by join date only needs an index covering inactive users — saving significant space versus indexing all 10,000+ users.

Verifying Index Effectiveness

```
db.users.find({ role: "trainee", track: "MERN Stack", isActive: true
}).explain("executionStats")
```

If the result shows `"stage": "IXSCAN"`, the compound index is being used. If it shows `"stage": "COLLSCAN"`, it means a full collection scan happened instead.

MODULE 7

MONGODB REFERENCE EN

TECHNICAL REFERENCE MANUAL

MongoDB Shell Command Reference

Complete Function List — Inputs and Return Values

Technical Reference • MongoDB / mongosh

Database-Level Commands

Function	Input	Return Value
show dbs	None	List of all databases with each one's size
use <dbName>	Database name	Switches the current database (text confirmation)
db	None	Name of the current database
show collections	None	List of all collections in the current database
db.dropDatabase()	None	Object containing ok: 1 and dropped: dbName
db.stats()	scale (optional)	Object with database statistics (size, collection count)
db.createCollection()	Collection name + options (optional)	Confirmation that the collection was created
db.getCollectionNames()	None	Array of all collection names

Create / Insert

Function	Input	Return Value
insertOne()	A single object (document)	Object containing acknowledged and insertedId
insertMany()	Array of objects	Object containing acknowledged and insertedIds (array of IDs)
insert() (Deprecated)	Object or array	Write result information (WriteResult)

Read / Find

Function	Input	Return Value
find()	Filter object (optional) + projection object (optional)	Cursor pointing to multiple documents
findOne()	Filter object (optional) + projection object (optional)	A single document, or null
countDocuments()	Filter object (optional)	Integer count of matching documents
estimatedDocumentCount()	None	Fast approximate count of all documents
distinct()	Field name + filter (optional)	Array of unique values for a given field

Update

Function	Input	Return Value
updateOne()	Filter object + update object (+ options)	Object with matchedCount and modifiedCount
updateMany()	Filter object + update object (+ options)	Object with matchedCount and modifiedCount
replaceOne()	Filter object + full replacement document	Object with matchedCount and modifiedCount
findOneAndUpdate()	Filter + update + options	The document (before or after update, per options) or null
findOneAndReplace()	Filter + replacement document + options	The old/new document, or null

Delete

Function	Input	Return Value
deleteOne()	Filter object	Object with deletedCount (0 or 1)
deleteMany()	Filter object	Object with deletedCount (integer)
findOneAndDelete()	Filter object (+ options)	The deleted document, or null
drop()	None	true on success, or an error if the collection does not exist

Cursor Modifiers

Function	Input	Return Value
sort()	Object specifying field and direction (1 or -1)	Cursor modified with the sort order
limit()	Integer	Cursor limited to a set number of results
skip()	Integer	Cursor after skipping a given number of results
toArray()	None	Array containing all documents
forEach()	Callback function	None — executes the function on each document
count() (Deprecated)	None	Count of documents in the cursor
hasNext()	None	true/false — whether a next document exists
next()	None	The next document in the cursor
pretty()	None	Same cursor with clearer formatting for display

Aggregation

Function	Input	Return Value
aggregate()	Array of pipeline stages	Cursor with the pipeline result

Indexes

Function	Input	Return Value
createIndex()	Object specifying field and type (+ options)	Name of the created index
createIndexes()	Array of index objects	Array of index names
getIndexes()	None	Array of all indexes on the collection
dropIndex()	Index name or field object	Confirmation of deletion
dropIndexes()	None	Confirmation that all indexes except <code>_id</code> were dropped
reIndex()	None	Rebuilds all indexes

Collection Info & Stats

Function	Input	Return Value
stats()	Options (optional)	Object with collection statistics (size, document count)
totalSize()	None	Total size in bytes
dataSize()	None	Data-only size in bytes
validate()	Options (optional)	Report on the structural integrity of the collection

Bulk Operations

Function	Input	Return Value
bulkWrite()	Array of operations (insert/update/delete)	Object summarizing all operations (insertedCount, modifiedCount, etc.)
initializeUnorderedBulkOp()	None	Bulk operation object (executed with no specific order)
initializeOrderedBulkOp()	None	Bulk operation object (executed in order)

Transactions

Function	Input	Return Value
<code>db.getMongo().startSession()</code>	None	Session object
<code>session.startTransaction()</code>	Options (optional)	Starts the transaction
<code>session.commitTransaction()</code>	None	Confirms and persists all transaction operations
<code>session.abortTransaction()</code>	None	Cancels all transaction operations
<code>session.endSession()</code>	None	Ends the session

User & Role Management

Function	Input	Return Value
db.createUser()	Object with user, pwd, roles	Confirmation that the user was created
db.dropUser()	Username	Confirmation of deletion
show users	None	List of all users on the database
db.grantRolesToUser()	Username + array of roles	Confirmation that privileges were granted
db.revokeRolesFromUser()	Username + array of roles	Confirmation that privileges were revoked

Watch / Change Streams

Function	Input	Return Value
watch()	Pipeline (optional) + options	Change stream cursor that monitors changes live